



UNIVERSITAS
INDONESIA



FAKULTAS
ILMU
KOMPUTER

Data Delivery (Markup Language & JSON)

Tim Dosen PBP

Our Previous Session...

- MTV Django
- TDD

Agenda

- How web communicates
- HTTP Requests and Responses
- Data Delivery
- HTML
- XML
- JSON
- API

History of the Web

By October of 1990, Tim Berners-Lee had written the three fundamental technologies that remain the foundation of today's web (and which you may have seen appear on parts of your web browser):

- HTML: HyperText Markup Language. The markup (formatting) language for the web.
- URI: Uniform Resource Identifier. A kind of “address” that is unique and used to identify to each resource on the web. It is also commonly called a URL.
- HTTP: Hypertext Transfer Protocol. Allows for the retrieval of linked resources from across the web.

HTML History Timeline

in the 1990s

- most web sites issued a whole page at a time
- clunky for users, excessive bandwidth

idea for improvement (HTML 2.0)

- update page incrementally
- do it asynchronously, so browser doesn't freeze

in 1997, HTML 4.0

in 1999, XMLHttpRequest arrives

- Microsoft invents XHR (XMLHttpRequest) idea for ActiveX in IE5
- XHR objects are used to interact with servers. You can retrieve data from a URL without having to do a full page refresh.
- adopted by other browsers for Javascript

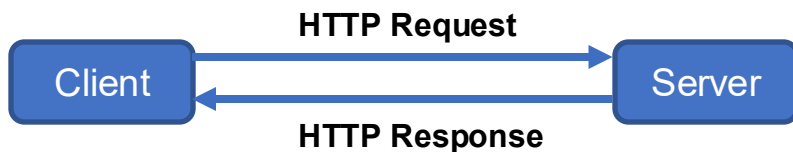
in 2005, called "Ajax"

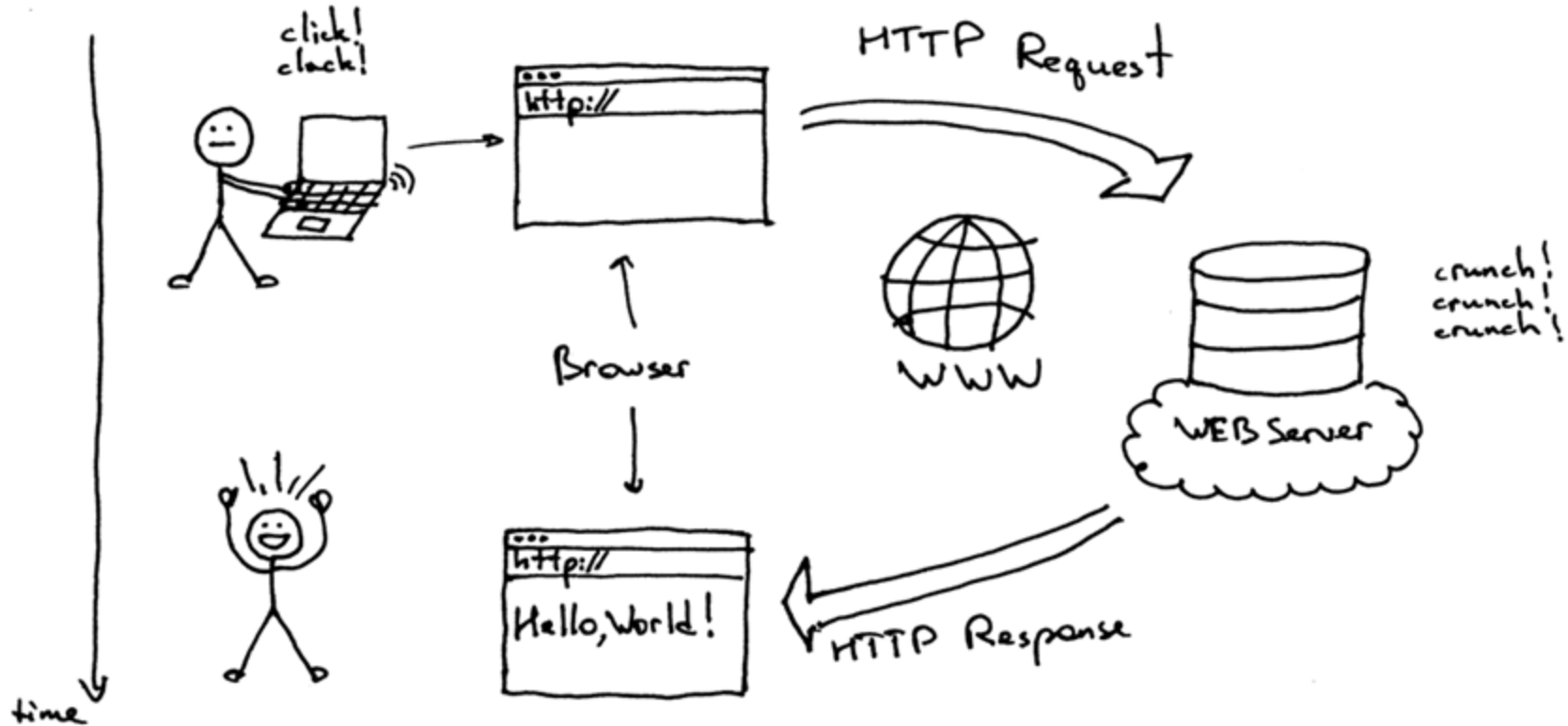
- asynchronous JavaScript and XML

in 2008, HTML 5.0

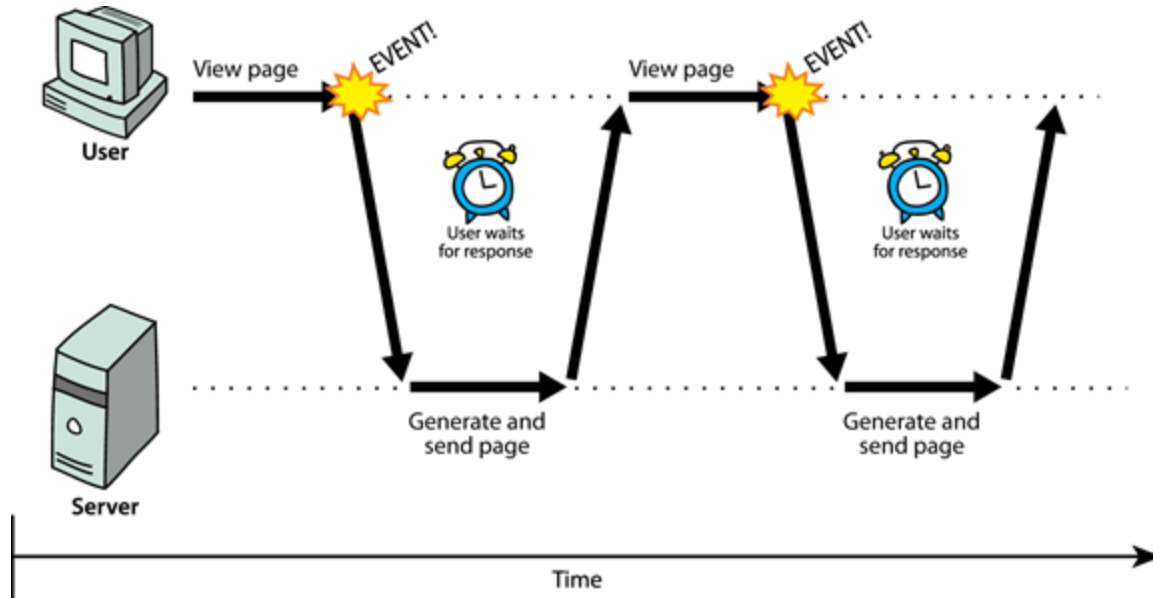
World Wide Web Communication

- The World Wide Web is about communication between web clients and web servers.
- Clients are often browsers (Chrome, Edge, Safari), but they can be any type of program or device.
- Servers are most often computers in the cloud.
- Communication between client computers and web servers is done by sending HTTP Requests and receiving HTTP Responses.



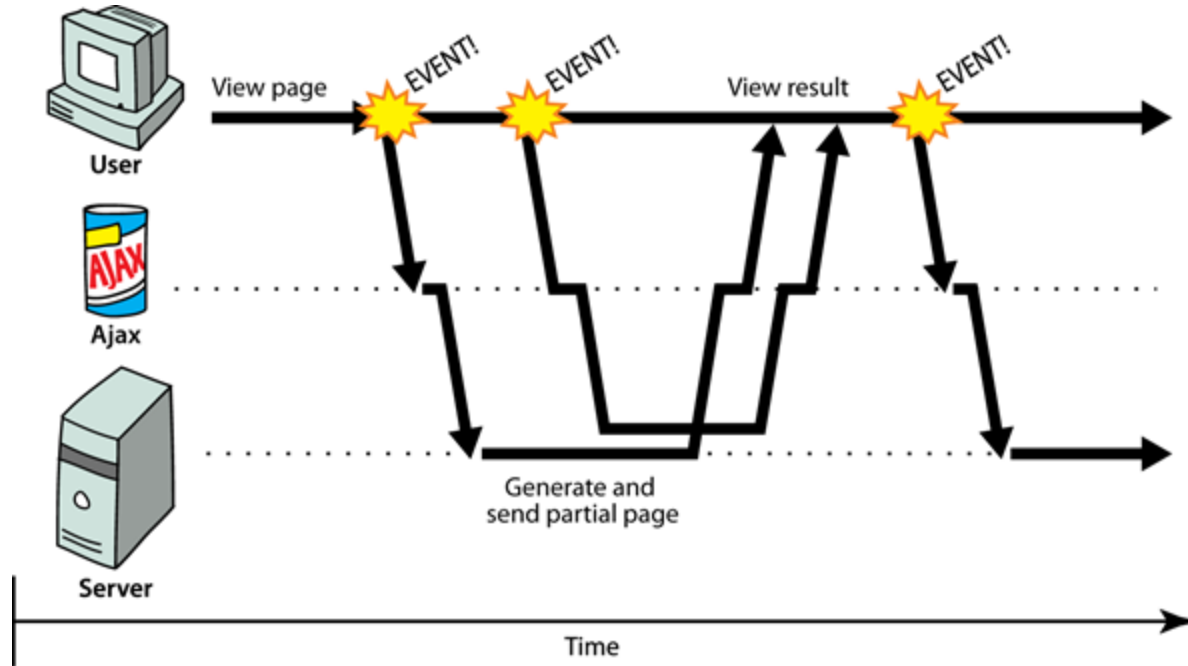


Synchronous Web Communication



user must wait while new pages load (click, wait, refresh)

Asynchronous Web Communication



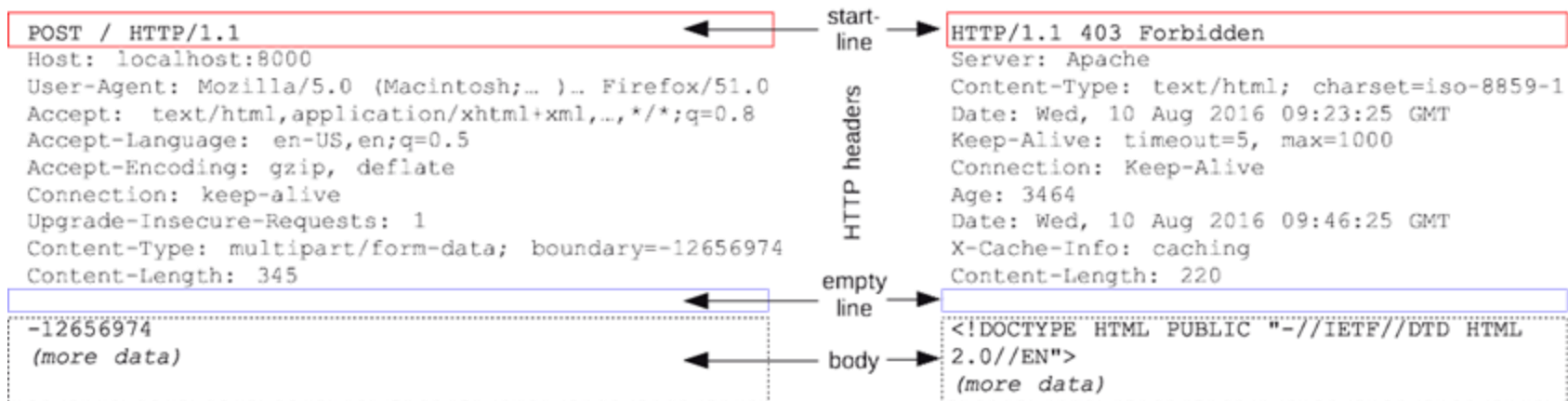
user can keep interacting with page while data loads

HTTP (Hyper Text Transfer Protocol) Requests & Responses

- HTTP requests and responses share similar structure and are composed of:
 - A start-line describing the requests to be implemented, or its status of whether successful or a failure. This start-line is always a single line.
 - An optional set of HTTP headers specifying the request, or describing the body included in the message.
 - A blank line indicating all meta-information for the request has been sent.
 - An optional body containing data associated with the request (like content of an HTML form), or the document associated with a response. The presence of the body and its size is specified by the start-line and HTTP headers.

Requests

Responses



HTTP Requests

- The most commonly used HTTP request methods are **GET**, **POST**, **PUT**, **PATCH**, and **DELETE**. These are equivalent to the **CRUD operations (create, read, update, and delete)**.
 - **GET**: GET request is used to read/retrieve data from a web server. GET returns an HTTP status code of **200 (OK)** if the data is successfully retrieved from the server.
 - **POST**: POST request is used to send data (file, form data, etc.) to the server. On successful creation, it returns an HTTP status code of **201**.
 - **PUT**: A PUT request is used to modify the data on the server. It replaces the entire content at a particular location with data that is passed in the body payload. If there are no resources that match the request, it will generate one.
 - **PATCH**: PATCH is similar to PUT request, but the only difference is, it modifies a part of the data. It will only replace the content that you want to update.
 - **DELETE**: A DELETE request is used to delete the data on the server at a specified location.

<https://www.geeksforgeeks.org/different-kinds-of-http-requests/>

HTTP Requests

```
POST / HTTP/1.1
```

```
Host: localhost:8000
```

```
User-Agent: Mozilla/5.0 (Macintosh;... )... Firefox/51.0
```

```
Accept: text/html,application/xhtml+xml,..., */*;q=0.8
```

```
Accept-Language: en-US,en;q=0.5
```

```
Accept-Encoding: gzip, deflate
```

```
Connection: keep-alive
```

```
Upgrade-Insecure-Requests: 1
```

```
Content-Type: multipart/form-data; boundary=-12656974
```

```
Content-Length: 345
```

```
-12656974
```

```
(more data)
```

Request headers

General headers

Representation
headers

HTTP Responses

- HTTP response status codes indicate whether a specific HTTP request has been successfully completed.
- Responses are grouped in five classes:
 - Informational responses (100–199)
 - Successful responses (200–299)
 - Redirection messages (300–399)
 - Client error responses (400–499)
 - Server error responses (500–599)

HTTP Responses

- Detail of HTTP Response Status

Code		Name	Description
1xx		informational	
2xx		Success	
	200	Ok	
3xx		Redirect	
	301	Moved permanently	
	302	Moved Temporarily	
4xx		Request Error	
	400	Bad request	Request can't understand by server.
	403	Forbidden	User is forbidden to perform request operation.
	404	Not found	Resource that requested by URi are not found.
	405	Method not allowed	Request methods are not allowed in a specific resource.
5xx		Server error	
	500	Internal server error	An error occurred on the server, the request could not be continued.
	503	Service unavailable	The server is not available, usually due to overload or maintenance

HTTP Responses

HTTP/1.1 200 OK

Access-Control-Allow-Origin: *

Connection: Keep-Alive

Content-Encoding: gzip

Content-Type: text/html; charset=utf-8

Date: Wed, 10 Aug 2016 13:17:18 GMT

Etag: "d9b3b803e9a0dc6f22e2f20a3e90f69c41f6b71b"

Keep-Alive: timeout=5, max=999

Last-Modified: Wed, 10 Aug 2016 05:38:31 GMT

Server: Apache

Set-Cookie: csrftoken=.....

Transfer-Encoding: chunked

Vary: Cookie, Accept-Encoding

X-Frame-Options: DENY

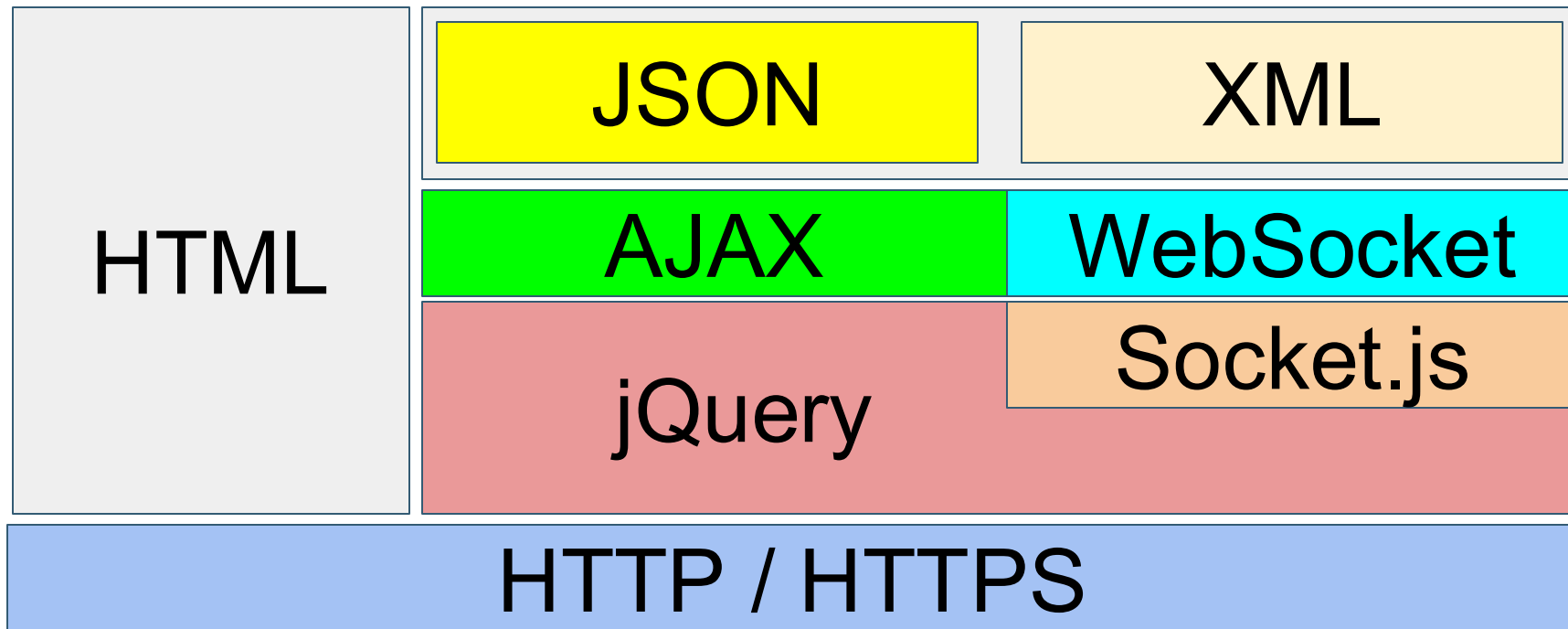
Response headers

Representation
headers

General headers

(body)

Data Delivery



Data Delivery

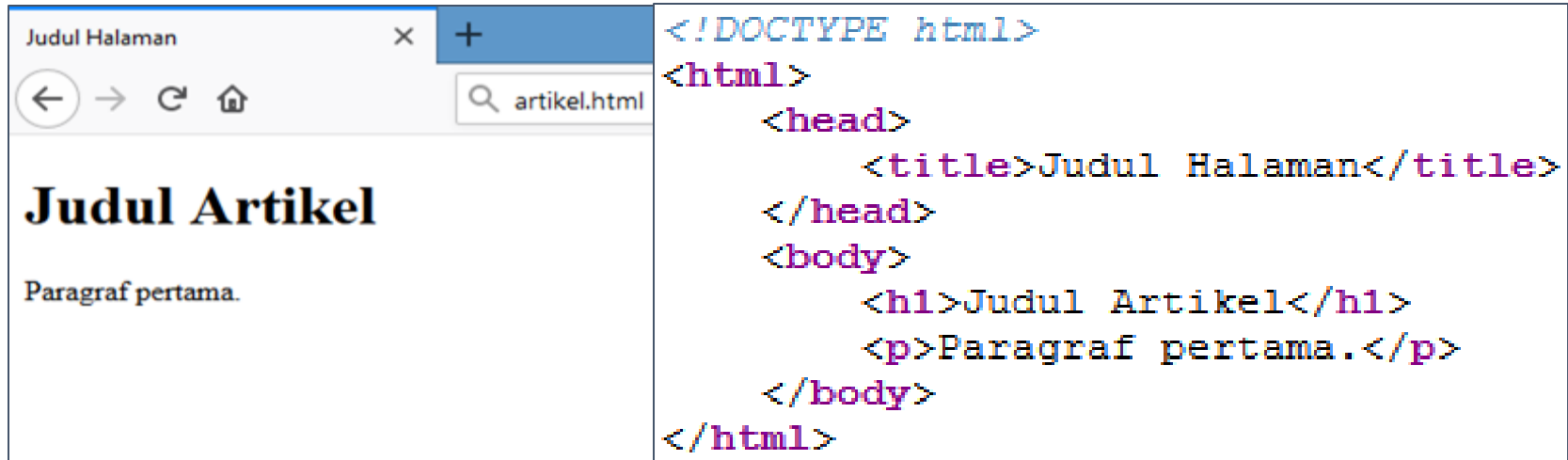
- A typical HTTP request / response circle:
 1. The browser requests an HTML page. The server returns an HTML file.
 2. The browser requests a style sheet. The server returns a CSS file.
 3. The browser requests a JPG image. The server returns a JPG file.
 4. The browser requests JavaScript code. The server returns a JS file.
 5. The browser requests data. The server returns data (in XML or JSON).
- Some files are not stored in the server, but it is generated by the code.

HTML (Hyper Text Markup Language)

- The HyperText Markup Language, or HTML, is the standard markup language for documents designed to be displayed in a web browser.
 - Markup Language is a system (such as HTML or SGML) for marking or tagging a document that indicates its logical structure (such as paragraphs) and gives instructions for its layout on the page especially for electronic transmission and display. (<https://www.merriam-webster.com/dictionary/markup%20language>)

HTML Page Structure

- HTML describes the structure of a web page semantically and originally included cues for the appearance of the document.
- HTML elements are the building blocks of HTML pages.



Judul Halaman ← → ↻ 🏠 🔍 artikel.html Judul Artikel Paragraf pertama.	<pre><!DOCTYPE html> <html> <head> <title>Judul Halaman</title> </head> <body> <h1>Judul Artikel</h1> <p>Paragraf pertama.</p> </body> </html></pre>
---	--

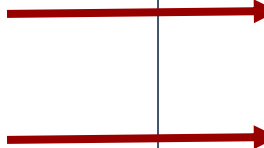
HTML Elements

`<title>Page Title</title>`

start tag element content end tag

Empty Elements

```
<body>
  <h1>Judul Artikel</h1>
  <p>Paragraf pertama.</p>
  <br>
  <p>Paragraf kedua.</p>
  <p>Paragraf ketiga.</p>
  <hr>
  <p>Paragraf keempat.</p>
</body>
```



HTML Tags

`<tagname>`content goes here...`</tagname>`

`<html>` : Defines the root of an HTML

`<head>` : Defines metadata/information about document
etc...

HTML Attributes

- Attributes provide additional information about elements. Attributes are always specified in the start tag.
- Attributes usually come in name/value pairs like: name="value"
- Example:

```
<section name="form">
  <h2>Send Message</h2>
  <form id="msg_form" method="POST" action="add_message">
    <input type="text" name="id_text" required>
    <button type="submit" class="btn btn-default">
      Submit
    </button>
  </form>
</section>
```

XML (eXtensible Markup Language)

```
<?xml version="1.0" encoding="UTF-8"?>
<note>
  <to>John</to>
  <from>Jane</from>
  <heading>Reminder</heading>
  <body>Don't forget me this weekend!</body>
</note>
```

- Definition: markup language and file format designed for storing, transmitting, and reconstructing data in a way that is both human-readable and machine-readable
- Some of XML Syntax Rules:
 - XML Documents Must Have a Root Element
 - The XML prolog is optional. If it exists, it must come first in the document.
 - All XML Elements Must Have a Closing Tag.
 - The XML prolog does not have a closing tag! This is not an error. The prolog is not a part of the XML document.
 - XML tags are case sensitive. The tag <Letter> is different from the tag <letter>. Opening and closing tags must be written with the same case.
 - XML does not truncate multiple white-spaces (HTML truncates multiple white-spaces to one single white-space).

Document Object Model (DOM)

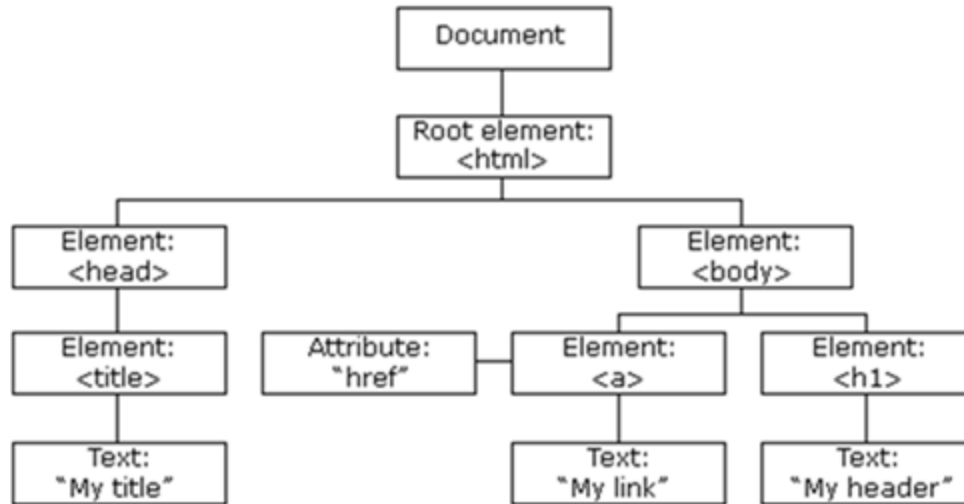
- The DOM defines a standard for accessing and manipulating documents:

"The W3C Document Object Model (DOM) is a platform and language-neutral interface that allows programs and scripts to dynamically access and update the content, structure, and style of a document."
- Understanding the DOM is a must for anyone working with HTML or XML.
- The HTML DOM defines a standard way for accessing and manipulating HTML documents. It presents an HTML document as a tree-structure.
- The XML DOM defines a standard way for accessing and manipulating XML documents. It presents an XML document as a tree-structure.
- Do you remember what is object?
 - A model class == a database table
 - A model instance == a database table row

HTML DOM

- When a web page is loaded, the browser creates a Document Object Model of the page.
- The HTML DOM model is constructed as a tree of Objects.

The HTML DOM Tree of Objects



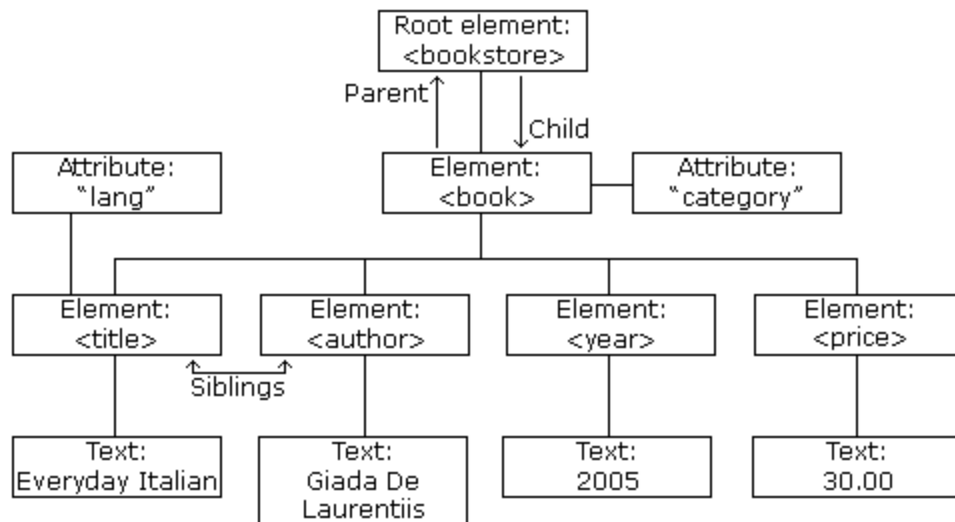
XML DOM

- The XML DOM views an XML document as a tree-structure. The tree structure is called a **node-tree**.
- In a node tree, the top node is called the root.
- Every node, except the root, has exactly one parent node.
- A node can have any number of children.
- A leaf is a node with no children.
- Siblings are nodes with the same parent.
 - The root node in the XML is named <bookstore>.
 - All other nodes in the document are contained within <bookstore>.
 - The root node <bookstore> holds 2 <book> nodes.
 - The first <book> node holds the child nodes: <title>, <author>, <year>, and <price>.
 - The child nodes contain one text node each, "Everyday Italian", "Giada De Laurentiis", "2005", and "30.00".

```
<?xml version="1.0" encoding="UTF-8"?>
<bookstore>
  <book category="cooking">
    <title lang="en">Everyday Italian</title>
    <author>Giada De Laurentiis</author>
    <year>2005</year>
    <price>30.00</price>
  </book>
  <book category="children">
    <title lang="en">Harry Potter</title>
    <author>J K. Rowling</author>
    <year>2005</year>
    <price>29.99</price>
  </book>
</bookstore>
```

XML DOM

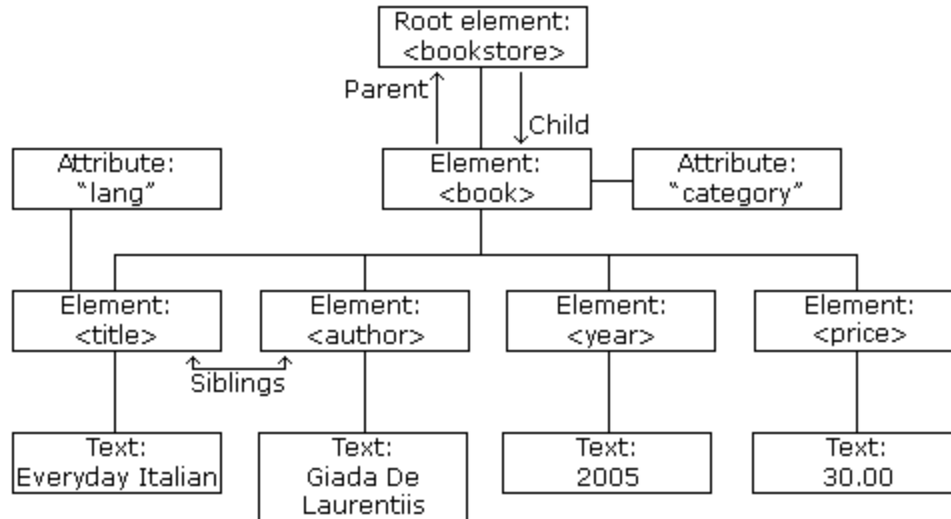
- The XML DOM views an XML document as a tree-structure. The tree structure is called a **node-tree**.



```
<?xml version="1.0" encoding="UTF-8"?>
<bookstore>
  <book category="cooking">
    <title lang="en">Everyday Italian</title>
    <author>Giada De Laurentiis</author>
    <year>2005</year>
    <price>30.00</price>
  </book>
  <book category="children">
    <title lang="en">Harry Potter</title>
    <author>J K. Rowling</author>
    <year>2005</year>
    <price>29.99</price>
  </book>
</bookstore>
```

XML DOM

- The XML DOM views an XML document as a tree-structure. The tree structure is called a **node-tree**.



- A common error in DOM processing is to expect an element node to contain text.
- However, the text of an element node is stored in a text node.
- In this example: `<year>2005</year>`, the element node `<year>` holds a text node with the value "2005".
- "2005" is not the value of the `<year>` element.

JSON (JavaScript Object Notation)

- Javascript is a scripting language that enables you to create dynamically updating content, control multimedia, animate images, and pretty much everything else.

```
<!DOCTYPE html>
<html>
<body>

<h2>Access a JavaScript Object</h2>
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML = "Kucing";
</script>

</body>
</html>
```

Source:

https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting/What_is_JavaScript

JSON (JavaScript Object Notation)

- JSON is a lightweight format for storing and transporting data.
- The JSON format is syntactically identical to the code for creating JavaScript objects.
- Because of this similarity, a JavaScript program can easily convert JSON data into native JavaScript objects.
- The JSON syntax is derived from JavaScript object notation syntax, but the JSON format is text only. Code for reading and generating JSON data can be written in any programming language.

JSON Syntax Rules

- Data is in name/value pairs
- Data is separated by commas
- Curly braces hold objects
- Square brackets hold arrays

```
{ "employees": [  
    { "firstName": "John", "lastName": "Doe" },  
    { "firstName": "Anna", "lastName": "Smith" },  
    { "firstName": "Peter", "lastName": "Jones" }  
  ] }
```

JSON vs XML

A syntax for storing and exchanging data
using XML

- Fetch an XML document
- Use the XML DOM to loop through the document
- Extract values and store in variables

using JSON

- Fetch a JSON string
- Use `JSON.parse()` to parse the JSON string
- is an easier to use alternative to XML.
- For AJAX applications, JSON is faster and easier than XML

JSON vs XML: Example

```
{  
  "employees": [  
    {  
      "firstName": "John",  
      "lastName": "Doe"  
    },  
    {  
      "firstName": "Anna",  
      "lastName": "Smith"  
    },  
    {  
      "firstName": "Peter",  
      "lastName": "Jones"  
    }  
  ]  
}
```

```
<employees>  
  <employee>  
    <firstName>John</firstName> <lastName>Doe</lastName>  
  </employee>  
  <employee>  
    <firstName>Anna</firstName> <lastName>Smith</lastName>  
  </employee>  
  <employee>  
    <firstName>Peter</firstName> <lastName>Jones</lastName>  
  </employee>  
</employees>
```

Work with JSON data

```
<!DOCTYPE html>
<html>
<body>

<h2>JSON Object Creation in JavaScript</h2>

<p id="demo"></p>

<script>
var text = '{"name":"John Johnson","street":"Oslo West 16","phone":"555 1234567"}'

var obj = JSON.parse(text);

document.getElementById("demo").innerHTML =
obj.name + "<br>" +
obj.street + "<br>" +
obj.phone;
</script>

</body>
</html>
```

- More: https://www.w3schools.com/js/js_json_objects.asp

Work with XML

- More:
https://www.w3schools.com/xml/dom_nodes_get.asp

```
<!DOCTYPE html>
<html>
<body>

<p id="demo"></p>

<script>
var xhttp = new XMLHttpRequest();
xhttp.onreadystatechange = function() {
    if (this.readyState == 4 && this.status == 200) {
        myFunction(this);
    }
};
xhttp.open("GET", "books.xml", true);
xhttp.send();

function myFunction(xml) {
    var xmlDoc = xml.responseXML;
    var x = xmlDoc.getElementsByTagName('title')[0];
    var y = x.childNodes[0];
    document.getElementById("demo").innerHTML = y.nodeValue;
}
</script>

</body>
</html>
```

Additional Materials

Application Programming Interface (API)

- APIs are the little pieces of code that make it possible for digital devices, software applications, and data servers to talk with each other, and they're the essential backbone of so many services we now rely on.
- Websites and APIs both do the same things, like return data, content, images, video, and other information, **but** APIs don't return all the details that are needed to make things look pretty for the human eye
- APIs return the raw data and other machine-readable information needed behind the scenes to put the resources being delivered to work, with very little assistance from a human.

How Do APIs Work?

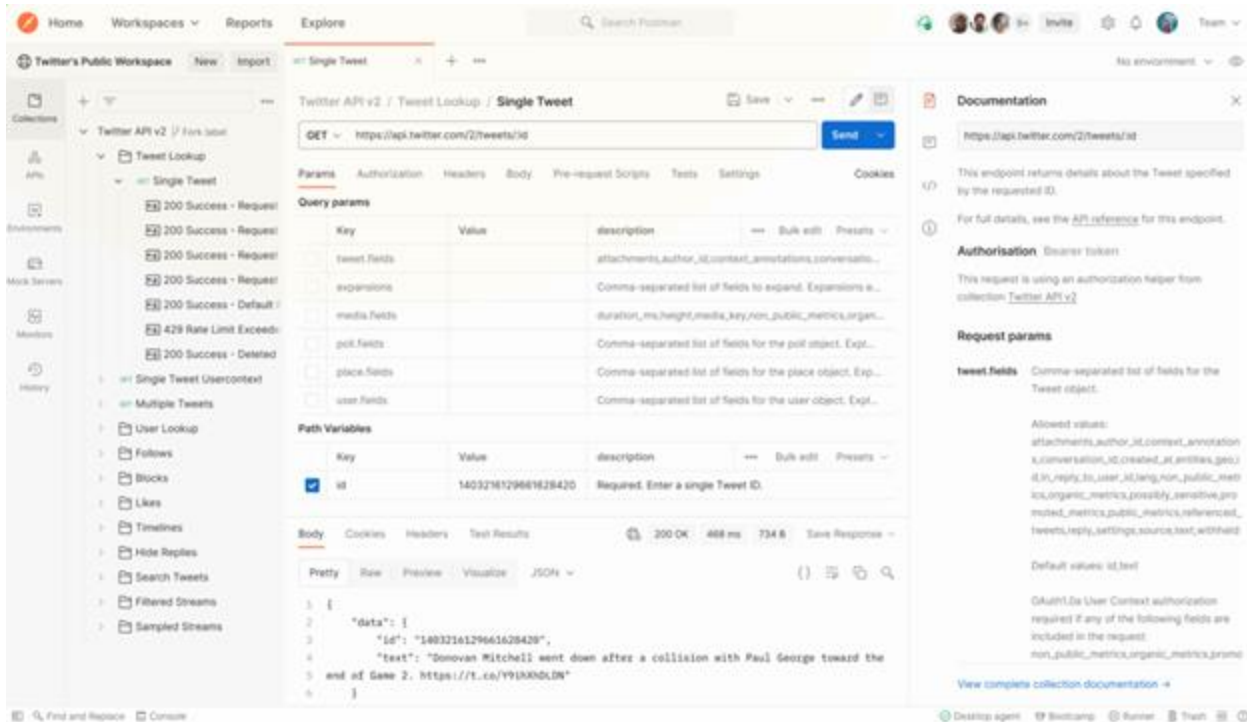
- APIs work by sharing data and information between applications, systems, and devices—making it possible for these things to talk with each other.
- A client application (like a mobile app) sends a request to the server → the server processes the request → the server returns a response to the client.
- A request includes the URL of the API endpoint and an HTTP request method. The method indicates the action you want the API to perform. Here are some of the most common methods:
 - GET retrieves data from an API.
 - POST sends new data to an API.
 - PATCH and PUT update existing data.
 - DELETE removes existing data.

Why Use APIs?

- **Integration with internal and external systems:** One of the top reasons developers use APIs is to integrate one system with another system.
- **Enabling mobile applications:** A lot of mobile applications rely on APIs to deliver important information to mobile users. For example, if you use your mobile phone to check in for a flight and select your seat, APIs can communicate the seat you selected so that flight attendants know where you are seated when you get onboard.
- **Improving organizational security and governance:** APIs can be used to improve organizational security. For example, APIs are often used to power Single Sign-On, which is the ability for users to use one username and password to login into multiple systems.

Postman

- <https://www.postman.com/>
- Postman is an API platform for building and using APIs.



Next Week



Form, Authentication, Session, and Cookie

Tim Pengajar PBP

References

- Django Documentation
 - <https://docs.djangoproject.com/en/5.0/>
- w3schools
 - https://www.w3schools.com/html/html_elements.asp
 - https://www.w3schools.com/html/html_attributes.asp
 - https://www.w3schools.com/js/js_json_objects.asp
 - <https://www.w3schools.com/tags/default.asp>
 - https://www.w3schools.com/whatis/whatis_http.asp
 - https://www.w3schools.com/whatis/whatis_json.asp
 - https://www.w3schools.com/whatis/whatis_htmlDOM.asp
 - https://www.w3schools.com/xml/xml_syntax.asp
 - https://www.w3schools.com/xml/dom_nodes.asp
 - https://www.w3schools.com/xml/dom_nodes_get.asp
 - <https://www.w3schools.com/xml/books.xml>
- APIs & Postman
 - <https://www.postman.com/>
 - <https://blog.postman.com/intro-to-apis-what-is-an-api/>
 - <https://learning.postman.com/docs/getting-started/sending-the-first-request/>
- <https://courses.cs.washington.edu/courses/cse154/12au/lectures/slides/lecture22-ajax.shtml>
- <https://developer.mozilla.org/en-US/docs/Web/HTTP/Messages>
- <https://www.merriam-webster.com/dictionary/markup%20language>